VPN Tunnel Health Monitoring & Alerting in Microsoft Intune — Scheduled Connectivity Health Checks, Telemetry & Alerts

Implementation Solution Guide

Version 1.0.0 · 2026-07-22

Contents

Executive Summary — VPN Tunnel Health Monitoring & Alerting in Microsoft Intune (Scheduled Connectivity Health Checks, Telemetry & Alerts) 4

1. Solution Overview — VPN Tunnel Health Monitoring & Alerting in Microsoft Intune (Scheduled Connectivity Health Checks, Telemetry & Alerts) 5

- What this solution is . . . 5
- This is MONITORING — read-only by contract, never a fix . . . 5
- What "healthy" means — the health signals . . . 6
- Where this sits — the explicit boundaries (what this is NOT) . . . 7
- How it works — the monitoring model . . . 7
- The standard deployment model . . . 8
- Benefits . . . 8
- Limitations (be honest with buyers) . . . 8

2. Requirements — VPN Tunnel Health Monitoring & Alerting in Microsoft Intune (Scheduled Connectivity Health Checks, Telemetry & Alerts) 9

- R1. Proactive Remediations (detection) entitlement — the scheduling engine . . . 9
- R2. A VPN profile already deployed to the target devices (not built here) . . . 10
- R3. Log Analytics workspace (for telemetry — optional but recommended) — **verify data path** . 10
- R4. Azure Monitor (for alerting — optional but recommended) — **verify alert wiring** . . . 10
- R5. Windows devices enrolled and managed (for the detection delivery) . . . 11
- R6. Intune Management Extension present on targets . . . 11
- R7. Device-side probe context and reachability . . . 11
- R8. A pilot device group . . . 11
- R9. A production target group . . . 11
- R10. An admin workstation with the Microsoft Graph PowerShell SDK . . . 12
- R11. Certificate visibility on the device (for the cert-expiry signal) . . . 12

Global Requirements — Intune Solutions (category-level) 12

- G1. Microsoft Entra ID tenant . . . 12
- G2. Microsoft Intune subscription . . . 12
- G3. Administrative roles . . . 13
- G4. Device enrollment in Intune . . . 13
- G5. Pilot security group . . . 13
- G6. Supported, current operating system . . . 13
- G7. Network access to required cloud endpoints . . . 13

3. Implementation — VPN Tunnel Health Monitoring & Alerting in Microsoft Intune (Scheduled Connectivity Health Checks, Telemetry & Alerts) 13

- Phase 1 — Define healthy and author the read-only health check . . . 14

Phase 2 — Package and scope to a pilot ring . . . 15
Phase 3 — Read results, wire telemetry, and build the dashboard . . . 16
Phase 4 — Validate, then expand to the fleet . . . 17
5. Testing & Validation — VPN Tunnel Health Monitoring & Alerting in Microsoft Intune (Scheduled Connectivity Health Checks, Telemetry & Alerts) 18
Test 0 — The health check makes NO changes (read-only proof) — do this first . . . 18
Test 1 — Package is present, assigned, reporting, and detection-only . . . 18
Test 2 — A healthy device reports Healthy . . . 19
Test 3 — A broken tunnel reports the correct red signal . . . 19
Test 4 — The fleet rollup shows the cohort . . . 19
Test 5 — An alert fires on a seeded failure . . . 20
Test 6 — Steady state (no thrash, still read-only) . . . 20
Acceptance gate . . . 20
6. Troubleshooting — VPN Tunnel Health Monitoring & Alerting in Microsoft Intune (Scheduled Connectivity Health Checks, Telemetry & Alerts) 20
The check reports a false RED (device is actually healthy) . . . 20
The check reports a false GREEN (device is actually broken) . . . 21
The script doesn't run at all (no status reported) . . . 21
Telemetry isn't arriving in Log Analytics . . . 22
The alert doesn't fire . . . 22
Per-VPN-type detection nuance . . . 22
Triage runbook — from a red signal to the layer that owns the fix . . . 23
Scripts section . . . 24
7. Best Practices — VPN Tunnel Health Monitoring & Alerting in Microsoft Intune (Scheduled Connectivity Health Checks, Telemetry & Alerts) 24
Keep monitoring and fixing separate . . . 25
Define "healthy" well . . . 25
Deploy & scope . . . 25
Telemetry & alerting . . . 26
Operate & govern . . . 26
8. Deliverables Summary — VPN Tunnel Health Monitoring & Alerting in Microsoft Intune (Scheduled Connectivity Health Checks, Telemetry & Alerts) 26
Documentation (this guide) . . . 26
PowerShell scripts (device probe + Microsoft Graph) — `VPNHEALTH-Scripts.zip` . . . 27
Hero image . . . 28
How it's delivered . . . 28
Appendix A — PowerShell Scripts 29

Executive Summary — VPN Tunnel Health Monitoring & Alerting in Microsoft Intune (Scheduled Connectivity Health Checks, Telemetry & Alerts)

Most teams find out a VPN tunnel is unhealthy the same way every time: a wave of **"I can't connect"** tickets. By then the outage is live — a client certificate expired overnight, a gateway stopped answering, a routing change quietly broke traffic to the one subnet that mattered, or DNS stopped resolving through the tunnel. There was **no proactive signal**.

This solution builds that signal. It turns VPN tunnel health into a **proactive, ongoing, fleet-wide monitoring practice deployed through Microsoft Intune** — across **Microsoft Tunnel**, **Windows Always On VPN** (device and user tunnel), and **third-party client VPNs** (e.g. Cisco Secure Client). It is the **endpoint-side health-monitoring layer**:

- **Define "healthy"** → concrete, checkable signals with pass/fail thresholds: VPN adapter/profile

present, tunnel established, gateway reachable, key routes and DNS resolving **through** the tunnel, and the auth certificate valid and **not near expiry**.

- **Deploy scheduled, read-only checks** → the health check ships as an Intune **Proactive Remediations

detection **script that runs on a schedule and reports each device's tunnel health. It is** read-only by contract** — it never restarts, reconnects, renews, or re-pushes a tunnel. This is monitoring, not remediation.

- **Roll up into telemetry** → results feed Intune reporting and (where wired) a **Log Analytics**

workspace, with **KQL** for a fleet health rollup by VPN type and gateway.

- **Alert on degradation** → an **Azure Monitor** alert turns a cert nearing expiry, a cohort failing to

establish, or an unreachable gateway into a notification — **before** the ticket wave.

- **Triage to the fix** → a runbook maps each red signal to the responsible layer and **points to the

product that owns the fix** — it never fixes the tunnel itself.

The boundaries are deliberate and explicit: this product **defers Microsoft Tunnel gateway-server health to the "Microsoft Tunnel Gateway" product**, **defers VPN deployment to the deployment products** (Microsoft Tunnel, Always On VPN, Cisco Secure Client), **defers reactive break/fix to the "Endpoint Network Connectivity Troubleshooting Toolkit"**, and **uses the "Proactive Remediations at Scale" detection model as its engine** without reproducing any of them.

Built the standard repeatable way — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — with exact, click-by-click Intune/Azure steps, a requirements guide that tells you how to verify *and* fix each prerequisite, five ready-to-run PowerShell scripts (a read-only device health check, three read-only Graph tools, and one deploy script that defaults to `-WhatIf` and only ever deploys the monitoring package), and testing/troubleshooting guides with clear pass/fail gates. Everything version- and vendor-specific — Proactive Remediations behavior, Log Analytics data paths, Azure Monitor alert wiring, per-vendor VPN detection, and national-cloud endpoints — is flagged **verify-with-Microsoft/vendor**.

Outcome: a proactive VPN tunnel health-monitoring practice run through Intune — scheduled checks, fleet telemetry, and alerts that tell you a tunnel is failing **before your users do**, across whichever VPN technologies you run — operated safely by your own team, without a consulting engagement.

1. Solution Overview — VPN Tunnel Health Monitoring & Alerting in Microsoft Intune (Scheduled Connectivity Health Checks, Telemetry & Alerts)

What this solution is

Most teams learn a VPN tunnel is unhealthy the same way every time: a wave of **"I can't connect"** tickets. By then the outage is already live — a client certificate expired overnight, a gateway stopped answering, a routing change quietly broke traffic to the one subnet that mattered, or DNS stopped resolving through the tunnel. There was **no proactive signal**.

This solution builds that signal. It turns VPN tunnel health into a **proactive, ongoing, fleet-wide monitoring practice deployed through Microsoft Intune**. It does four things:

1. **Defines "healthy"** for a VPN tunnel as concrete, checkable signals with pass/fail thresholds.

2. **Deploys scheduled, read-only health checks** to your managed devices using Intune's **Proactive

Remediations detection model**, so every device reports its own tunnel health on a schedule.

1. **Rolls the results up into telemetry** — Intune reporting and (where available) a Log Analytics

workspace you can query with KQL and dashboard by VPN type and gateway.

1. **Alerts** when a tunnel degrades — a certificate nearing expiry, a *cohort* of devices failing to

establish, a gateway going unreachable — so you act **before** the ticket wave.

It is **VPN-technology-aware**: the same practice spans **Microsoft Tunnel**, **Windows Always On VPN** (both the device tunnel and the user tunnel), and **third-party client VPNs** (for example Cisco Secure Client / AnyConnect). The health-check script adapts what it probes per VPN type; the telemetry and alerting sit *above* the VPN technology so one dashboard covers all of them.

This is MONITORING — read-only by contract, never a fix

The single most important thing to understand about this package: **the health checks never touch the tunnel.** They are **read-only probes**. They observe adapter state, connection state, reachability, DNS, and certificate validity, and they **report** — they do not restart a service, reconnect an adapter, renew a certificate, or re-push a profile. This is deliberate:

• A monitor that "fixes while it checks" hides the real failure rate and can itself cause harm at

scale (imagine every device in a failing cohort simultaneously bouncing its adapter).

• Monitoring must be **safe to run everywhere, always** — the only way to guarantee that is to make it

incapable of changing anything.

Fixes are **out of scope here and deferred, with explicit pointers**, to the deployment and troubleshooting products (see "Where this sits" below). If you want automated *remediation* of a condition this monitor detects, that is a separate, deliberately-authored remediation built with the safety discipline of the **"Proactive Remediations at Scale — Detection & Remediation Script Packs for Microsoft Intune"** product — never a side effect of the health check.

What "healthy" means — the health signals

The health-check script evaluates a tunnel against these signals and returns a single **health verdict** (Healthy / Degraded / Unhealthy) plus a per-signal breakdown. Each signal has a pass/fail rule:

Signal	Healthy (pass)	Not healthy (fail)
VPN adapter / profile present	The expected VPN profile/adapter for the device's VPN type exists.	Profile/adapter missing — the VPN was never provisioned or was removed.
Tunnel established / connected	The tunnel reports a connected/established state.	Disconnected, dialing, or error state.
Gateway / endpoint reachable	The VPN head-end/gateway endpoint responds.	Gateway unreachable (timeout / no route).
Key routes reachable through the tunnel	A defined internal test target is reachable *via the tunnel*.	The internal target is unreachable while the tunnel is up (routing/split-tunnel problem).
DNS resolves through the tunnel	An internal name resolves using the tunnel's DNS.	Internal name fails to resolve (DNS not flowing through the tunnel).
Auth certificate valid & not near expiry	The auth certificate is present, valid, and **more than the warning window** (default **30 days**) from expiry.	**Warn** inside 30 days; **Fail** if expired or missing.

You set the concrete values — the internal **test route** (e.g. an internal host/IP), the internal **DNS name** to resolve, the certificate **subject/issuer** to check, and the **cert-expiry warning window** (default 30 days) — once, at the top of the health-check script (Implementation §Step 1).

> ***VPN-type awareness (flag per-vendor specifics).*** *How each signal is *read* differs by VPN technology, and the exact detection surface is **vendor- and version-specific — verify against current Microsoft/vendor documentation**: - **Microsoft Tunnel** — profile/connection state surfaced through the Microsoft Defender for Endpoint client on the supported platforms; the health check probes reachability, route, DNS, and the auth certificate the profile uses. - **Windows Always On VPN (device tunnel + user tunnel)** — Windows exposes the VPN connection objects and status natively (the script can enumerate the connection and its state); a device tunnel and a user tunnel are checked as *separate* connections. - **Third-party client VPNs (e.g. Cisco Secure Client / AnyConnect)** — presence and state are read from the vendor client's adapter/service and status surface; the exact service name, adapter name, and status API are **vendor-specific and must be verified against the vendor's documentation.***

Where this sits — the explicit boundaries (what this is NOT)

This product is intentionally narrow: it is the **endpoint-side health-monitoring layer**. It does **not** deploy VPNs and it does **not** own gateway-server health. State these boundaries plainly so buyers don't expect the wrong thing:

- **Microsoft Tunnel gateway-SERVER health belongs to another product.** The **"Microsoft Tunnel

Gateway — Deployment, VPN Configuration & Ongoing Operations" **solution OWNS the** server-side **health and status of a Microsoft Tunnel gateway in Intune (the Linux host, the container/agent, the Site/Server Configuration, HA, and the server-side health blade).** This **product** defers all server-side health to it **and focuses on** endpoint-side **scheduled checks — "can this *device* reach and use its tunnel?" — plus** cross-VPN** telemetry and alerting that no single deployment product provides. (Referenced, not reproduced.)

- **VPN *deployment* belongs to the deployment products.** Standing up each VPN — the Microsoft Tunnel

gateway and profiles, Windows **Always On VPN** (device + user tunnel), or **Cisco Secure Client (AnyConnect) VPN Deployment via Microsoft Intune** — is owned by those products. This product **assumes a VPN profile is already deployed** and monitors it. (Referenced, not reproduced.)

- **Reactive, deep break/fix belongs to the toolkit product.** When a red signal needs hands-on

diagnosis, the **"Endpoint Network Connectivity Troubleshooting Toolkit (DNS, Proxy, VPN, IP/Port/URL Filtering)"** product is the reactive break/fix engine. This product's triage runbook **points to it** rather than reproducing it.

- **The Proactive Remediations *mechanics* belong to that product.** This package **uses** the

Proactive Remediations *detection* model as its scheduling/reporting engine but does **not** re-teach how Proactive Remediations works, how to license it, or how to author remediation pairs — that is the **"Proactive Remediations at Scale — Detection & Remediation Script Packs for Microsoft Intune"** product (referenced as the engine, not reproduced). Here we deploy a **detection-only** health check — there is no remediation script.

How it works — the monitoring model

1. **Define healthy.** You set the health signals and thresholds (test route, internal DNS name,

certificate subject/issuer, expiry warning window) once at the top of the health-check script.

1. **Deploy the read-only check on a schedule.** You package the health-check script as a **Proactive

Remediations detection script **(there is** no** remediation script — this is monitoring) and assign it to a device group with a recurrence (e.g. daily). Intune's management extension runs it on each device on that cadence.

1. **Each device reports its verdict.** The script writes a structured, human-readable health line and

signals overall health through the **exit-code contract** (`exit 0` = healthy; `exit 1` = degraded/ unhealthy) so the Intune console shows a per-device with-issue / without-issue split.

1. **Telemetry rolls it up.** Where a **Log Analytics** workspace is wired in, the health results are

queryable with **KQL**, so you can build a **fleet health rollup by VPN type and gateway**. (Whether Proactive Remediations output is natively queryable in Log Analytics, and the exact data path, **must be verified against current Microsoft documentation** — see Requirements and Implementation.)

1. **Alerts fire on degradation.** An **Azure Monitor** alert rule on the telemetry (a KQL result

crossing a threshold) turns a degraded/expiring tunnel into a notification — cert nearing expiry, a cohort failing to establish, a gateway unreachable. (Exact alert wiring is **verify-with-Microsoft**.)

1. **Triage points to the fix.** When a signal goes red, the triage runbook maps it to the responsible

layer and **points to the product that owns the fix** — it never fixes the tunnel itself.

The standard deployment model

This solution uses **one** repeatable model so the process is recognizable across the catalog:

> ***Define the health signals and thresholds → author the read-only, VPN-type-aware detection health check → package it as a named Proactive Remediations detection (no remediation) → scope to a pilot ring → wire telemetry (Log Analytics/KQL) → build the fleet health dashboard → configure alert rules → validate against seeded failures → expand to the fleet in waves.***

Every step follows the repeatable pattern: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

Benefits

- **Know before your users do** — a scheduled health signal replaces the ticket wave; you see a tunnel

failing on a cadence, fleet-wide, instead of finding out from the help desk.

- **One practice across every VPN you run** — Microsoft Tunnel, Always On VPN (device + user tunnel),

and third-party client VPNs roll up into one health view, by VPN type and gateway.

- **Catch certificate expiries early** — the cert-expiry warning window turns a silent, dated failure

into an alert weeks ahead, not a Monday-morning outage.

- **Safe by construction** — the checks are **read-only**; they can run everywhere, always, without

ever risking the tunnel they watch.

- **Fast, correct triage** — a red signal maps straight to the responsible layer and the product that

owns the fix, so time-to-diagnosis drops.

- **Reuses tools you already have** — the Proactive Remediations detection engine, Intune reporting,

and (optionally) Log Analytics/Azure Monitor; no new agent to deploy.

Limitations (be honest with buyers)

- **It monitors; it does not fix.** By design, the health checks make **no** changes to the tunnel.

Remediation is deferred to the deployment/troubleshooting products (with pointers). This is a feature, not a gap.

• **A VPN must already be deployed.** This package assumes each device already has its VPN profile; standing up the VPN belongs to the deployment products.

• **Licensing & feature availability gate the engine.** The Proactive Remediations detection model requires the licensing/enrollment that product documents — **verify current licensing and feature availability against Microsoft's documentation** (Requirements §R1).

• **Telemetry & alerting depend on optional Azure plumbing.** The fleet KQL rollup and Azure Monitor alerts require a **Log Analytics workspace** and **Azure Monitor**, and the exact data path from Proactive Remediations output into Log Analytics **must be verified against current Microsoft documentation** (Requirements §R3–R4). Without them you still get per-device Intune reporting; you lose the cross-VPN KQL dashboard and threshold alerts.

• **Scheduling is periodic, not instant** — detection runs on a recurrence, so a health signal is "eventually", not real-time. Exact cadence limits are **verify-with-Microsoft**.

• **Per-vendor detection specifics vary.** How each signal is read for a given VPN client (service name, adapter name, status surface) is **vendor- and version-specific and must be verified against the vendor's documentation** before you rely on it.

2. Requirements — VPN Tunnel Health Monitoring & Alerting in Microsoft Intune (Scheduled Connectivity Health Checks, Telemetry & Alerts)

Confirm **every** requirement below before you begin. Each entry follows the same pattern: **Description · Why required · How to verify · How to implement if missing.** Category-wide prerequisites (tenant, Intune licensing, admin roles, device enrollment, supported OS, network) are covered once in the **Global Requirements — Intune Solutions** document; the items here are specific to deploying and operating an endpoint-side VPN tunnel health-monitoring practice.

> ***Verify-with-Microsoft flag.*** *Proactive Remediations licensing and behavior, the data path from Proactive Remediations output into* ***Log Analytics****, Intune Data Collection/Reports,* ***Azure Monitor*** *alert wiring, and console navigation* ***change over time and differ by cloud (Commercial vs. GCC / GCC High / DoD)****. Where this document states a licensing SKU, a portal path, a data path, or a behavior, treat it as a starting point and* ***verify it against current Microsoft documentation for your tenant and cloud*** *before relying on it. Per-vendor VPN client detection specifics must be verified against the* ***vendor's*** *documentation. Facts you must not assume are called out inline.*

R1. Proactive Remediations (detection) entitlement — the scheduling engine

Description: A license/entitlement that lets the tenant create and assign **Endpoint Analytics Proactive Remediations** (device health scripts), which this solution uses to run the read-only health check on a schedule. **Why:** The health check is delivered and scheduled as a Proactive Remediations **detection** script; without the entitlement the **Remediations** experience is unavailable. **Verify:** In Intune, open **Reports** → **Endpoint analytics** → **Remediations** (path may differ by tenant — **verify against current Microsoft documentation**). If you can create a script package, the entitlement is present. **Implement:**

Acquire/assign the licensing Microsoft currently requires for Endpoint Analytics Proactive Remediations (**verify the exact SKU with Microsoft** — Commercial vs. GCC/GCC High/DoD may differ). The mechanics of Proactive Remediations are covered by the **"Proactive Remediations at Scale — Detection & Remediation Script Packs for Microsoft Intune"** product (referenced, not reproduced).

R2. A VPN profile already deployed to the target devices (not built here)

Description: The devices you will monitor **already have a VPN profile** — Microsoft Tunnel, Windows Always On VPN (device and/or user tunnel), or a third-party client VPN (e.g. Cisco Secure Client) — assigned and functional. **Why:** This product **monitors** an existing tunnel; it does not deploy one. With no VPN profile present, every device correctly reports "adapter/profile missing" and the monitor has nothing to watch. **Verify:** On a target device, confirm the expected VPN profile/connection exists and can connect; in Intune, confirm the VPN configuration profile is assigned to the device. **Implement:** Deploy the VPN first using its owning product — **"Microsoft Tunnel Gateway — Deployment, VPN Configuration & Ongoing Operations"**, the **Windows Always On VPN** product(s), or **"Cisco Secure Client (AnyConnect) VPN Deployment via Microsoft Intune"** (referenced, not reproduced). Return here once devices have a working VPN profile.

R3. Log Analytics workspace (for telemetry — optional but recommended) — verify data path

Description: A Microsoft Azure **Log Analytics workspace** — **named** `LA – Endpoint – VPNHealth` — to receive the health telemetry for KQL rollups and dashboards. **Why:** Per-device results are visible in Intune's Remediations reporting without any extra plumbing; the **cross-VPN fleet rollup by type and gateway** and the **threshold alerts** need the results in a queryable store (Log Analytics), where the included KQL runs. **Verify:** In the Azure portal, **Log Analytics workspaces** → `LA – Endpoint – VPNHealth` exists. Then confirm the **data path** actually lands your health data there — **whether Proactive Remediations output is natively queryable in Log Analytics, and by which table/connector, must be verified against current Microsoft documentation.** If a native path is unavailable in your tenant/cloud, the health check can ship a structured line that a Log Analytics data-collection path ingests (verify the supported ingestion method for your environment). **Implement:** Azure portal → **Create a resource** → **Log Analytics workspace**; name it `LA – Endpoint – VPNHealth`. Wire the health-result data path per Microsoft's current guidance for your cloud (**verify** — do not assume a specific table name). Without this workspace you still get per-device Intune reporting; you lose the KQL fleet dashboard and alerts.

R4. Azure Monitor (for alerting — optional but recommended) — verify alert wiring

Description: Azure Monitor enabled on the `LA – Endpoint – VPNHealth` workspace so a KQL result can raise an alert. **Why:** Alerting turns a degraded/expiring tunnel into a notification (cert nearing expiry, a cohort failing to establish, a gateway unreachable). Azure Monitor **scheduled query (log) alert rules** evaluate the telemetry on a cadence and fire when a threshold is crossed. **Verify:** Azure portal → **Monitor** → **Alerts** → **Alert rules**; you can create a **log search alert** scoped to `LA – Endpoint – VPNHealth`. **Implement:** Create the alert rule in Implementation §Step 8 (**named** `ALERT – VPN Tunnel Health – Degraded`). The exact alert-rule type, evaluation cadence, threshold options, and action-group wiring **change over time and must be verified against current Microsoft documentation** for your cloud.

R5. Windows devices enrolled and managed (for the detection delivery)

Description: The target devices are **Windows** devices enrolled in Intune (MDM-managed or co-managed with the workload directed to Intune, per Microsoft's current support statement). **Why:** The Proactive Remediations detection engine delivers the health-check script to enrolled Windows endpoints via the Intune management extension. **Platform support for Proactive Remediations must be verified against current Microsoft documentation. Verify:** Intune → **Devices** → **All devices**; targets appear, are **Windows**, and are compliant. **Implement:** Enroll the devices (per Global Requirements G4) and confirm they check in.

R6. Intune Management Extension present on targets

Description: The **Intune Management Extension (IME)** agent is installed and healthy on the target devices. **Why:** IME is what actually runs the PowerShell detection health check on the device; if it is absent or unhealthy, the check never runs and no health status returns. **Verify:** On a target, confirm the **Microsoft Intune Management Extension** service is present and running (IME installs automatically when a device receives a PowerShell script or Win32 app assignment). **Exact service name/behavior — verify against current Microsoft documentation. Implement:** Assign any Intune PowerShell script or Win32 app to trigger IME installation, then re-check.

R7. Device-side probe context and reachability

Description: The health check runs in the correct **run context** (typically **system**, machine context, 64-bit) and the device can actually attempt the tunnel/route/DNS probes it performs. **Why:** The probes read machine-level VPN state and attempt reachability to an internal test target through the tunnel; run-context and the probe targets must be chosen so a *healthy* device passes and an *unhealthy* one fails — not the reverse. A wrong run context can make the script read nothing and falsely report a problem (or miss one). **Verify:** Run the health check locally on one healthy pilot device in the intended context and confirm it returns **Healthy**; run it on a device with the tunnel down and confirm it returns the correct red signal. (See Testing.) **Implement:** Package the detection to run as **system** unless a user-context VPN (e.g. an Always On **user** tunnel) requires the **logged-on user's** context; set **Run this script using the logged-on credentials** accordingly (Implementation §Step 4). **Confirm the current run-context options and their exact behavior against Microsoft's documentation.**

R8. A pilot device group

Description: A Microsoft Entra security group — **named** `PILOT – VPN Health – Devices` — containing a small set of representative devices that already have a VPN profile. **Why:** No monitor goes fleet-wide unproven; the pilot ring is where you confirm the health check reports correctly (healthy passes, seeded failures go red) before you widen. **Verify:** Entra → **Groups**; `PILOT – VPN Health – Devices` exists with a handful of pilot devices. **Implement:** Entra → **Groups** → **New group** → **Security**; name it `PILOT – VPN Health – Devices` and add the pilot devices (Implementation §Step 3).

R9. A production target group

Description: A Microsoft Entra security group — **named** `VPN Health – Windows – Production` — representing the broader fleet you will eventually monitor. **Why:** After the pilot passes you widen to

production in waves; a named production group is the assignment target for that expansion. **Verify:** Entra → **Groups**; `VPN Health – Windows – Production` exists (it may start empty and fill in waves). **Implement:** Entra → **Groups** → **New group** → **Security**; name it `VPN Health – Windows – Production`.

R10. An admin workstation with the Microsoft Graph PowerShell SDK

Description: A workstation with the **Microsoft.Graph** PowerShell SDK for the included discovery, validation, deploy, and reporting scripts. **Why:** The discovery/validation/report scripts query Intune via Microsoft Graph (read-only); the one deploy script creates/assigns the health-check package via Graph. **Verify:** `Get-Module Microsoft.Graph -ListAvailable` returns the SDK; `Connect-MgGraph` succeeds. **Implement:** `Install-Module Microsoft.Graph -Scope CurrentUser`. The scripts request read scopes such as `DeviceManagementConfiguration.Read.All` (and …`ReadWrite.All` for the deploy script) and consent on first connect. **Graph endpoints/scopes for Proactive Remediations (device health scripts) are on the beta endpoint and may change — verify against current Microsoft documentation.**

R11. Certificate visibility on the device (for the cert-expiry signal)

Description: The health check can read the **auth certificate** the VPN uses (subject/issuer and `NotAfter`) from the device certificate store. **Why:** The cert-expiry signal is one of the highest-value early warnings; it needs to locate the VPN auth certificate and compute days-to-expiry against the warning window (default **30 days**). **Verify:** On a pilot device, confirm the VPN's client-auth certificate is present in the expected store (typically **LocalMachine\My** for device/system context) and readable. **Implement:** Set the certificate **subject or issuer match** at the top of the health-check script (Implementation §Step 1). For a **user**-context certificate, run the check in the user context (R7). The exact store/location depends on how the VPN's certificate was provisioned (SCEP/PKCS) — **verify against the deploying product's configuration.**

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level on the marketplace so individual solution listings stay small. Confirm these are in place before deploying *any* Intune solution.

Each entry: **Description · Why required · How to verify · How to implement if missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, enrollment, and security policies. **Verify:** `https://intune.microsoft.com` loads and **Tenant administration** → **Tenant status** shows an active subscription. **Implement:** Assign Microsoft 365 E3/E5,

Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies, enrollment connectors, and configuration profiles requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator** (or Global Administrator for connector setup). **Implement:** Assign **Intune Administrator** to the operator; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted devices are enrolled and manageable in Intune. **Why:** Policies only reach enrolled (or app-managed) devices, regardless of platform. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Configure the appropriate enrollment method for the platform and ownership model, then enroll the device.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted to a small group before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists with intended members. **Implement:** Entra → **Groups** → **New group** → type **Security** → add pilot members.

G6. Supported, current operating system

Description: A supported OS baseline with current updates for the targeted platform. **Why:** Modern Intune features and security settings assume a patched, vendor-supported OS. **Verify:** Check the device's OS version or Intune → device → **Hardware**. **Implement:** Deploy update policies or upgrade to a supported release.

G7. Network access to required cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Implementation — VPN Tunnel Health Monitoring & Alerting in Microsoft Intune (Scheduled Connectivity Health Checks, Telemetry & Alerts)

Follow these steps **in order**. The *why* lives in the Solution Overview; the steps below are short and exact so you can **click to implement**. The design is: **define the health signals and thresholds → author the read-only, VPN-type-aware detection health check → package it as a named Proactive Remediations detection (no remediation) → scope to a pilot ring → wire telemetry (Log**

Analytics/KQL) → build the fleet health dashboard → configure alert rules → validate against seeded failures → expand to the fleet in waves. Pilot first: **Assess → Configure → Pilot → Validate → Expand.**

Consoles used:

- **Microsoft Intune admin center** — `https://intune.microsoft.com`
- **Microsoft Entra admin center** — `https://entra.microsoft.com`
- **Microsoft Azure portal** — `https://portal.azure.com` (Log Analytics + Azure Monitor)

> ***Read-only, by contract.*** *Every device-side probe in this solution is* ***read-only****. Nothing in this implementation restarts, reconnects, renews, or re-pushes a VPN. There is* ***no remediation script*** *— this is a monitoring package. Fixes are deferred to the deployment/troubleshooting products named in the Triage runbook (Troubleshooting).*

> ***Navigation & data-path flag.*** *The exact* ***Remediations*** *location in Intune, the* ***Log Analytics*** *data path for Proactive Remediations output, and the* ***Azure Monitor*** *alert-rule experience change over time and differ by cloud (Commercial vs. GCC / GCC High / DoD). This guide uses the common paths and* ***flags every place you must verify against current Microsoft documentation*** *before relying on it.*

Phase 1 — Define healthy and author the read-only health check

Step 1. Define the health signals and thresholds (configure the script header)

1. Open the included `VPNHEALTH-Detect-TunnelHealth.ps1` — the read-only, VPN-type-aware device

health check. You will deploy this file **as-is** as the detection script; you only edit the **configuration block at the top**.

1. Set these values for your environment (each maps to a health signal in the Overview):

- `$VpnType` — `MicrosoftTunnel`, `AlwaysOnDevice`, `AlwaysOnUser`, or `ThirdParty` (the script

adapts what it probes per type). For a third-party client, also set the vendor **adapter/service name** — **verify the exact name against the vendor's documentation.**

- `$ExpectedProfileName` — the VPN profile/connection name to look for (adapter/profile-present

signal).

- `$TestRouteTarget` — an **internal** host/IP that should be reachable **through the tunnel**

(key-route signal), e.g. `10.20.0.10`.

- `$TestDnsName` — an **internal** DNS name that should resolve through the tunnel (DNS signal),

e.g. `intranet.contoso.com`.

- `$CertSubjectMatch` / `$CertIssuerMatch` — a substring identifying the VPN **auth

certificate** (cert signal).

- `$CertWarningDays` — the cert-expiry **warning window** (default `30`). Inside this window

→ **Degraded (warn)**; expired/missing → **Unhealthy (fail)**.

1. Save the file. **Do not** add any code that changes the device — the check is read-only by contract.

Step 2. Confirm the verdict + exit-code contract

1. The health check emits a single structured **health line** (surfaced in the console's detection output) and communicates overall health through the **exit-code contract**:

- `exit 0` = **Healthy** — all signals pass (cert may be Healthy).
- `exit 1` = **Degraded or Unhealthy** — one or more signals warn/fail (including cert inside the warning window). In the Proactive Remediations console this shows the device as **With issue**.

1. Because this is a **detection-only** package, "With issue" means **"this device's tunnel is not fully healthy — investigate"**, *not* "a remediation will run". There is no paired remediation.

Phase 2 — Package and scope to a pilot ring

Step 3. Create the pilot and production groups

1. Entra → **Groups** → **New group** → **Security**; **name it** `PILOT – VPN Health – Devices` and add a small set of representative devices that already have a VPN profile (Requirements §R2, §R8).

1. Entra → **Groups** → **New group** → **Security**; **name it** `VPN Health – Windows – Production` (it may start empty; you fill it in waves during Expand — §R9).

Step 4. Package the health check as a Proactive Remediations detection — name it

1. In Intune, go to **Reports** → **Endpoint analytics** → **Remediations** → **Create script package** (*verify the current path per the navigation flag above; some tenants surface it under **Devices** → **Remediations***).

1. **Basics: name it** `VPNHEALTH – Tunnel Health – Detect – Pilot`; add a clear description and the **publisher** (your team name — white-label; do not brand it).

1. **Settings:**

- **Detection script file:** upload `VPNHEALTH-Detect-TunnelHealth.ps1` (Step 1).
- **Remediation script file: leave empty.** This is a **detection-only** monitor — do **not** attach a remediation. (If your tenant requires a remediation file, attach a no-op that only writes a log line and `exit 0` — it must make **no** change. Fixes belong to the deployment/troubleshooting products, not to this monitor.)
- **Run this script using the logged-on credentials: No** (system context) for Microsoft Tunnel, Always On **device** tunnel, and most third-party checks; set **Yes** only when monitoring a user-context VPN such as an Always On **user** tunnel or a user-store certificate (Requirements §R7).
- **Enforce script signature check** and **Run script in 64-bit PowerShell:** set per your signing

posture and the target's needs. **Verify current option names/behavior against Microsoft's documentation.**

1. **Review + create.**

Step 5. Assign to the pilot ring and schedule

1. Open `VPNHEALTH – Tunnel Health – Detect – Pilot` → **Assignments** → **Select groups to include** →

choose `PILOT – VPN Health – Devices`.

1. Set the **schedule** for the assignment. Start with **Daily** so you get a clean daily health signal.

(Available cadences — Once / Hourly / Daily — and their exact behavior should be **verified against current Microsoft documentation**.)

1. **Save.** The read-only health check now runs on the pilot devices on that schedule and reports each

device's verdict back to the console. **No device is changed** — this is monitoring.

Phase 3 — Read results, wire telemetry, and build the dashboard

Step 6. Read per-device health in the console

1. Open `VPNHEALTH – Tunnel Health – Detect – Pilot` → the **Overview / Device status** view.

2. Read the columns Endpoint Analytics reports (names may vary — **verify current column labels**):

• **Detection status** — **Without issue** = Healthy; **With issue** = Degraded/Unhealthy.

• **Detection output** — the structured health line your script wrote (which signals passed/failed,

and cert days-to-expiry).

1. Confirm the pilot splits into Healthy vs. not as expected. Run the included

`VPNHEALTH-Discovery.ps1` and `VPNHEALTH-Validate.ps1` for a tenant-side view (the package exists, is assigned, is running, and is reporting; telemetry path present).

Step 7. Wire telemetry into Log Analytics (optional but recommended) — verify the data path

1. In the Azure portal, confirm the workspace `LA – Endpoint – VPNHealth` exists (Requirements §R3).

2. Route the health results into the workspace. **The exact data path for Proactive Remediations output

into Log Analytics — whether it is natively queryable, and via which table/connector — must be verified against current Microsoft documentation for your tenant and cloud.** Use whichever of these Microsoft currently supports for your environment:

• **Intune → Log Analytics data path** (Intune reporting / data collection into a workspace), if

Proactive Remediations results are included; **verify** the table name your data lands in.

• A **device data-collection path** that ingests the structured health line the check emits, if a

native path is unavailable. Do **not** assume a specific table name — confirm it, then use that name in the KQL (Step 9).

1. Save a **named saved query** in the workspace — `VPN Tunnel Health – Fleet Rollup` — using the

KQL below (Step 9) once you have confirmed the correct source table.

Step 8. Configure the alert rule (optional but recommended) — verify the alert wiring

1. Azure portal → **Monitor** → **Alerts** → **Create** → **Alert rule**; **scope** it to the workspace

`LA – Endpoint – VPNHealth`.

1. **Condition:** a **log search (scheduled query) alert** whose query returns the degraded cohort (for

example: devices reporting Unhealthy, or certs inside the warning window, grouped so a *cohort* threshold — not a single device — triggers). Set the threshold and evaluation cadence.

1. **Details: name the rule** `ALERT – VPN Tunnel Health – Degraded`; attach an **action group**

(email/Teams/ITSM) for the notification.

1. **Create.** The alert now fires when the tunnel health telemetry crosses your threshold — cert

nearing expiry, a cohort failing to establish, or a gateway unreachable.

> ***Flag:*** *the alert-rule type, evaluation cadence, threshold/grouping options, and action-group wiring* ***change over time and must be verified against current Microsoft documentation*** *for your cloud before you rely on them.*

Step 9. Build the fleet health rollup (KQL)

Use this KQL as the `VPN Tunnel Health – Fleet Rollup` saved query and as the alert condition base. **Replace** `VpnTunnelHealth_CL` **with the actual table your data lands in (Step 7) — verify it; do not assume this name.** The same query ships as a comment in `VPNHEALTH-Report.ps1`.

```
// VPN Tunnel Health – Fleet Rollup (by VPN type and gateway)
// NOTE: table name is a placeholder — set it to the table your health data lands in (verify per Step 7).
VpnTunnelHealth_CL
| summarize arg_max(TimeGenerated, *) by DeviceName_s        // latest verdict per device
| summarize
    Devices        = count(),
    Healthy        = countif(Verdict_s == "Healthy"),
    Degraded       = countif(Verdict_s == "Degraded"),
    Unhealthy      = countif(Verdict_s == "Unhealthy"),
    CertExpiringSoon = countif(CertDaysToExpiry_d <= 30 and CertDaysToExpiry_d >= 0)
    by VpnType_s, Gateway_s
| extend UnhealthyPct = round(100.0 * Unhealthy / Devices, 1)
| sort by Unhealthy desc, Degraded desc
```

Pin the result to an **Azure dashboard** (or an Intune/Workbook view) so the fleet health rollup by VPN type and gateway is visible at a glance.

Phase 4 — Validate, then expand to the fleet

Step 10. Validate against seeded failures

Run the Testing guide against the pilot ring before widening: a healthy device reports **Healthy**; a device with a broken tunnel (adapter down / route unreachable / cert near expiry) reports the correct red signal; the **fleet rollup** shows the cohort; an **alert fires** on a seeded failure; and the checks made **no change** to any tunnel. Do not expand until the acceptance gate passes.

Step 11. Expand to production in waves

1. Only after the pilot passes, create the production assignment. Recommended: **clone** the pilot

package to a production copy **named** `VPNHEALTH – Tunnel Health – Detect – Production` (or re-assign the same package to the production group — keep pilot and production distinguishable).

1. Assign `VPNHEALTH – Tunnel Health – Detect – Production` to `VPN Health – Windows – Production`,

and fill that group in **waves** (a first wave, validate, then widen) rather than the whole fleet at once.

1. Keep the schedule conservative (Daily), watch the fleet rollup and the alert, and widen only as each

wave stays clean. Because the check is read-only, expansion carries no risk to the tunnels — but waves keep the telemetry/alert signal clean and interpretable.

5. Testing & Validation — VPN Tunnel Health Monitoring & Alerting in Microsoft Intune (Scheduled Connectivity Health Checks, Telemetry & Alerts)

Validate against the **pilot ring** (`PILOT – VPN Health – Devices`) before expanding. Each test has an explicit **PASS/FAIL** gate. Use the included PowerShell scripts (Microsoft Graph) for tenant-side checks and the on-device / console / Azure steps for behavior. The package under test is `VPNHEALTH – Tunnel Health – Detect – Pilot` built in Implementation.

> ***Set up the positive and negative cases first.*** *Keep at least one pilot device fully* ***healthy*** *(VPN profile present, tunnel connected, cert well within validity) as the negative control. Then, on other pilot devices,* ***seed*** *each failure below so the monitor has something to catch. All seeding is on the *device/VPN* side; the health check itself changes nothing.*

Test 0 — The health check makes NO changes (read-only proof) — do this first

Procedure: On one pilot device, record the VPN state (profile present, connection status, adapter state, certificate). Run `VPNHEALTH-Detect-TunnelHealth.ps1` locally (or let one scheduled cycle run). Re-record the same state. **PASS:** The VPN state is **identical** before and after — the tunnel was not restarted, reconnected, renewed, or reconfigured; only the log file under `C:\ProgramData\VPNHealth\` was written. The monitor is read-only. **FAIL:** Any change to the tunnel/adapter/certificate state → stop; the script has been modified to do more than observe. Restore the read-only script before proceeding.

Test 1 — Package is present, assigned, reporting, and detection-only

Procedure: Run `VPNHEALTH-Discovery.ps1`, then `VPNHEALTH-Validate.ps1`. **PASS:** `VPNHEALTH – Tunnel Health – Detect – Pilot` exists, is assigned to `PILOT – VPN Health – Devices`, is

returning run status (devices reporting), and has **no remediation script attached** (read-only monitor). **FAIL:** Missing / unassigned / no status → fix the assignment and schedule (Implementation §Step 4–5), confirm the Intune Management Extension is healthy (Requirements §R6), then re-run. A remediation script present → remove it; this monitor must be detection-only.

Test 2 — A healthy device reports Healthy

Procedure: After a detection cycle, open the package → **Device status**, or read the healthy device's detection output line. **PASS:** The healthy control device reports **Without issue** (exit 0) and its output line reads `verdict=Healthy` with every signal passing. Detection did **not** change the device. **FAIL:** A genuinely healthy device reports **With issue** → a false red. Check the configuration block (right `$VpnType`, `$ExpectedProfileName`, `$TestRouteTarget`, `$TestDnsName`, `$CertSubjectMatch`) and the run context (Requirements §R7); the probe targets must be reachable from a healthy device.

Test 3 — A broken tunnel reports the correct red signal

Seed **one** break per device and confirm the specific signal (not just "something's wrong"):

- **Adapter/profile down** — disconnect or remove the VPN profile → expect `verdict=Unhealthy` with `ProfilePresent=False` and/or `TunnelConnected=False`.
- **Route unreachable through the tunnel** — block/withdraw the internal `$TestRouteTarget` route while the tunnel is up → expect `verdict=Unhealthy` with `RouteThroughTunnel=False`.
- **DNS not through the tunnel** — make `$TestDnsName` fail to resolve → expect `DnsThroughTunnel=False`.
- **Gateway unreachable** — point at an unreachable gateway (or block it) → expect `GatewayReachable=False`.
- **Cert near expiry** — target a certificate inside the **30-day** warning window → expect `verdict=Degraded` with `CertValid=False` and `certDaysToExpiry` ≤ 30. **PASS:** Each seeded device reports the **matching** red signal in its detection output, and the verdict is `Unhealthy` (hard signals) or `Degraded` (cert warning). The healthy control stays Healthy. **FAIL:** Wrong or missing signal → check that the seeded condition matches the configured probe target, and the run context (Requirements §R7). No device was changed by the check.

Test 4 — The fleet rollup shows the cohort

Procedure: Run `VPNHEALTH-Report.ps1` (Graph or `-Source Local`), and — if telemetry is wired — run the `VPN Tunnel Health – Fleet Rollup` KQL in `LA – Endpoint – VPNHealth`. **PASS:** The rollup groups devices **by VPN type and gateway** and shows the seeded failures as a counted cohort (Unhealthy/Degraded/CertExpiringSoon counts reflect the seeded devices); the healthy control counts as Healthy. **FAIL:** The cohort is missing → confirm the report source has data (allow a cycle), and — for the KQL — that the **table name matches the actual data path** (Implementation §Step 7, **verify-with-Microsoft**).

Test 5 — An alert fires on a seeded failure

Procedure: With `ALERT – VPN Tunnel Health – Degraded` configured (Implementation §Step 8), keep a seeded cohort failing across an evaluation cycle. **PASS:** The alert rule fires and the action group delivers the notification (email/Teams/ITSM) for the seeded cohort — for example, the cert-expiry or gateway-unreachable condition raises the alert. **FAIL:** No alert → confirm the alert query returns the seeded cohort, the threshold/grouping is set so a cohort (not a single transient device) triggers, and the action group is attached. The exact alert-rule wiring is **verify-with-Microsoft** for your cloud.

Test 6 — Steady state (no thrash, still read-only)

Procedure: Over several cycles, watch the healthy control and a device you have since **repaired** (using the appropriate deployment/troubleshooting product — not this monitor). **PASS:** The healthy device stays **Healthy** every cycle; the repaired device flips from red to **Healthy** on the next cycle after *you* fixed it — evidence the monitor tracks reality without acting. Across all cycles, no device's tunnel state was changed by the check. **FAIL:** A device flaps Healthy/Unhealthy with no underlying change → tighten a probe target that is intermittently reachable (e.g. a flaky test route) so the signal reflects real health, not probe noise.

Acceptance gate

Proceed to **Expand** (production waves) only when **Tests 0–6 pass**: the check is proven **read-only**, the package is assigned and reporting **detection-only**, a **healthy device reports Healthy**, each **seeded break reports the correct red signal**, the **fleet rollup shows the cohort**, an **alert fires on a seeded failure**, and steady state shows **no thrash** with **no tunnel changes**. Anything less stays in the pilot ring until it passes.

6. Troubleshooting — VPN Tunnel Health Monitoring & Alerting in Microsoft Intune (Scheduled Connectivity Health Checks, Telemetry & Alerts)

Work top-down: confirm the check **runs at all** (agent + context), then that it reports the **right verdict** (no false red/green), then that **telemetry** reaches Log Analytics, then that **alerts** fire, and finally use the **Triage runbook** to route a genuine red signal to the layer — and the product — that owns the fix. Remember: this is a **read-only monitor**; it never changes the tunnel, so "fixing" always happens in another product.

> *Portal paths, the Log Analytics data path, Azure Monitor alert wiring, and Microsoft Graph **beta** endpoints below change over time and by cloud — **verify against current Microsoft documentation** for your tenant. Per-vendor VPN client detection specifics must be verified against the **vendor's** documentation.*

The check reports a false RED (device is actually healthy)

- **Wrong VPN type or profile name.** If `$VpnType` or `$ExpectedProfileName` doesn't match the device's

actual VPN, `ProfilePresent`/`TunnelConnected` read false on a healthy device. Set them to match the deployed VPN and re-run `VPNHEALTH-Detect-TunnelHealth.ps1` locally.

• **Probe target unreachable *by design*.** `$TestRouteTarget` or `$TestDnsName` points at something a

healthy device genuinely can't reach (wrong host, not published through the tunnel, or a split-tunnel exclusion). Choose an internal target that a healthy tunnel *should* reach.

• **Run-context mismatch.** A **user**-tunnel or user-store certificate read from **system** context (or

vice-versa) reads nothing and looks failed. Match the package's **Run this script using the logged-on credentials** to what the check touches (Requirements §R7).

• **Third-party adapter/service name wrong.** For `ThirdParty`, the `$ThirdPartyAdapter` must match the

vendor's actual adapter/service name — **verify with the vendor's documentation**; a near-miss reads as "not present".

• **Cert match too narrow/broad.** If `$CertSubjectMatch`/`$CertIssuerMatch` doesn't select the real VPN

auth certificate, `CertValid` fails on a healthy device. Confirm the cert's subject/issuer in the device store (Requirements §R11).

The check reports a false GREEN (device is actually broken)

• **Probe target too lenient.** A `$TestRouteTarget` that is reachable *without* the tunnel (e.g. also

routable on the local LAN) passes even when the tunnel is down. Pick a target only reachable **through** the tunnel so the route signal means what you think.

• **Cert window too short.** A certificate expiring in, say, 20 days still passes if `$CertWarningDays`

is set below that. Set the warning window to at least your renewal lead time.

• **Checking the wrong connection.** On Always On, a **device** tunnel can be healthy while the **user**

tunnel is down (or vice-versa). Monitor each as its own $VpnType/profile so neither masks the other.

The script doesn't run at all (no status reported)

• **Intune Management Extension missing/unhealthy.** IME executes the check on-device; if it isn't

installed or is broken, nothing runs and no status returns. Confirm the **Microsoft Intune Management Extension** service is present and running (Requirements §R6); assigning any Intune PowerShell script/Win32 app installs IME.

• **Not assigned / not yet scheduled.** Confirm `VPNHEALTH – Tunnel Health – Detect – Pilot` is assigned

to `PILOT – VPN Health – Devices` with a schedule (Implementation §Step 5) and allow a cycle. Run `VPNHEALTH-Validate.ps1`.

• **Signing/execution policy blocked it.** If your org enforces script signing, an unsigned check fails

to run. Sign the script or set the run/signing options Microsoft documents for your case — **verify current behavior**.

• **Wrong run context for the data it reads.** A check that reads user-context VPN state assigned as

system may run but read nothing (looks like a failure). Set the context per Requirements §R7.

Telemetry isn't arriving in Log Analytics

- **Data path not wired / wrong table.** Whether Proactive Remediations output is natively queryable in

Log Analytics, and via which table/connector, is **verify-with-Microsoft**. Confirm the data path from Implementation §Step 7 is actually configured for your tenant/cloud, and that the KQL uses the **real** table name your data lands in (the shipped KQL uses a **placeholder** table name — replace it).

- **Reporting/ingestion delay mistaken for "no data".** Both Intune reporting and Log Analytics ingest on

a delay; allow the interval before concluding nothing arrived. Cross-check with `VPNHEALTH-Report.ps1` (`-Source Graph`) or the on-device logs (`-Source Local`) which don't depend on the workspace.

- **Workspace scope/permissions.** Confirm `LA – Endpoint – VPNHealth` exists and the query is scoped to

it, and that whatever identity/agent ships the data has permission to write to it.

The alert doesn't fire

- **Query returns nothing for the seeded cohort.** Run the alert's KQL directly in `LA – Endpoint –

VPNHealth`; if it returns no rows for the seeded failure, the alert can't fire. Fix the query/table name first (see telemetry above).

- **Threshold/grouping set for single devices, not cohorts.** If the rule requires a large cohort but you

seeded one device, it won't trigger. For testing, lower the threshold; for production, set it so a real cohort (or a cert-expiry window) triggers without alerting on every transient.

- **Action group not attached / wrong target.** `ALERT – VPN Tunnel Health – Degraded` must have a valid

action group (email/Teams/ITSM). Confirm it, and send a test notification.

- **Evaluation cadence.** Scheduled-query alerts evaluate on an interval; allow at least one evaluation

window. Exact rule type, cadence, and options are **verify-with-Microsoft** for your cloud.

Per-VPN-type detection nuance

- **Microsoft Tunnel** — client state is surfaced via the Microsoft Defender for Endpoint client on the

supported platforms; if the profile/connection isn't enumerable the way the script expects, **verify the current client and its status surface against Microsoft's documentation**, and lean on the reachability/route/DNS/cert signals which are technology-agnostic.

- **Always On VPN (device vs user)** — enumerate device-tunnel and user-tunnel connections **separately**;

a user tunnel only exists/reads when a user is signed in — an offline device legitimately shows the user tunnel absent. Interpret accordingly, don't alert on "no user tunnel while logged off".

• **Third-party (e.g. Cisco Secure Client)** — presence/state come from the vendor's adapter/service; the

exact names and status API are **vendor-specific — verify against the vendor's documentation**. If the vendor exposes no scriptable status, rely on the reachability/route/DNS/cert signals as the health proxy.

Triage runbook — from a red signal to the layer that owns the fix

This monitor **does not fix**; it tells you **where** to fix. Map the red signal to the responsible layer, then use the product that owns that fix (referenced, not reproduced here):

Red signal	Most likely layer	Where to fix (product to use)
`ProfilePresent=False` (adapter/profile missing)	**VPN profile / deployment**	The VPN's **deployment** product — **"Microsoft Tunnel Gateway — Deployment, VPN Configuration & Ongoing Operations"**, the **Windows Always On VPN** product(s), or **"Cisco Secure Client (AnyConnect) VPN Deployment via Microsoft Intune"** — re-push/repair the profile.
`TunnelConnected=False` (won't establish), single device	**Device / client**	Reactive break/fix on the device — **"Endpoint Network Connectivity Troubleshooting Toolkit (DNS, Proxy, VPN, IP/Port/URL Filtering)"**.
`TunnelConnected=False` across a **cohort** on one gateway	**Gateway / head-end (server side)**	Gateway-**server** health — **"Microsoft Tunnel Gateway — Deployment, VPN Configuration & Ongoing Operations"** (it OWNS server-side health; this monitor defers to it). For a third-party head-end, the vendor/appliance owner.
`GatewayReachable=False`	**Gateway / network path**	Gateway-server health (Tunnel Gateway product) or your **network/firewall** owner; confirm the head-end is up and the path/ports are open.
`RouteThroughTunnel=False` (tunnel up, internal target unreachable)	**Routing / split-tunnel config**	The VPN **deployment** product's routing/Server Configuration (e.g. include-routes on Microsoft Tunnel) — fix the routes.

Red signal	Most likely layer	Where to fix (product to use)
`DnsThroughTunnel=False`	**DNS / VPN DNS config**	The VPN **deployment** product's DNS settings, then deep DNS diagnosis with the **Connectivity Troubleshooting Toolkit**.
`CertValid=False` — near expiry or expired	**Certificate / PKI (SCEP/PKCS)**	Renew/re-provision via the certificate pipeline the VPN uses (the **deployment** product's SCEP/PKCS configuration); this is the highest-value early warning — act inside the window.

If you decide a detected condition warrants **automated** remediation, author it deliberately as its own Proactive Remediation with the safety discipline of **"Proactive Remediations at Scale — Detection & Remediation Script Packs for Microsoft Intune"** — never by adding a fix to this read-only monitor.

Scripts section

- **Connect-MgGraph scopes.** `VPNHEALTH-Discovery.ps1`, `VPNHEALTH-Validate.ps1`, and

`VPNHEALTH-Report.ps1` are **read-only** and request `DeviceManagementConfiguration.Read.All`; `VPNHEALTH-Deploy-HealthCheck.ps1` requests …`ReadWrite.All` + `Group.Read.All` **only** when run with `-Execute` (its default dry-run uses read scopes). If a call returns nothing, confirm consent to the scope and that the account has the Intune role (Global Requirements G3).

- **Exact-name matching.** The Graph scripts match objects by **exact display name** — reuse

`VPNHEALTH – Tunnel Health – Detect – Pilot`, `PILOT – VPN Health – Devices`, and the other object names **verbatim**. A renamed object won't be found; pass `-PackageName`/`-PilotGroupId` to override.

- **Read-only nature.** The device check and all three Graph query scripts make **no** changes. The only

state-changer, `VPNHEALTH-Deploy-HealthCheck.ps1`, changes **Intune objects only** (the monitoring package + assignment), **never a VPN**, and **defaults to** `-WhatIf` — it prints the plan and changes nothing until you re-run with `-Execute`. It always deploys the package **detection-only** (no remediation), preserving the read-only-monitoring guarantee.

- **Beta endpoints.** Proactive Remediations = `deviceManagement/deviceHealthScripts` on the Graph

beta endpoint; run states / detection output property names can change — **verify against current Microsoft documentation** if a script returns nothing.

7. Best Practices — VPN Tunnel Health Monitoring & Alerting in Microsoft Intune (Scheduled Connectivity Health Checks, Telemetry & Alerts)

Field-proven guidance for running a proactive, endpoint-side VPN tunnel health-monitoring practice that stays trustworthy — and stays **read-only**.

Keep monitoring and fixing separate

- **The health check observes; it never acts.** The single most important rule: the detection health check is **read-only** — it must never restart, reconnect, renew, or re-push a VPN. A monitor that "fixes while it checks" hides the real failure rate and can cause harm at scale. Keep all fixes out of this package.
- **Deploy it as detection-only.** Package the health check as a Proactive Remediations **detection** with **no remediation script** attached. "With issue" here means *"investigate this device's tunnel"*, not *"a fix will run."*
- **Send fixes to the products that own them.** When a signal goes red, route to the deployment or troubleshooting product (Triage runbook) — never bolt a fix onto the monitor. If you genuinely want automated remediation of a detected condition, author it deliberately as its own Proactive Remediation with the safety discipline of the **"Proactive Remediations at Scale"** product.

Define "healthy" well

- **Pick stable, representative probe targets.** The internal `$TestRouteTarget` and `$TestDnsName` should be things that are *supposed* to be reachable through the tunnel and are themselves reliable — a flaky test target produces false reds and alert fatigue.
- **Check each VPN type on its own terms.** Set `$VpnType` correctly; a Windows Always On **device** tunnel and **user** tunnel are separate connections and are monitored separately. For third-party clients, confirm the exact adapter/service name against the **vendor's** documentation.
- **Tune the cert-expiry window to your renewal lead time.** The default 30-day warning should be at least as long as it takes you to renew/re-provision a VPN certificate. If renewal is slow, widen the window so the alert gives you room.

Deploy & scope

- **Pilot ring first, every time.** Assign the monitor to `PILOT – VPN Health – Devices` and prove it (healthy passes, seeded breaks go red) before production — no exceptions.
- **Seed real failures in the pilot.** Deliberately break the tunnel/route/DNS/cert on pilot devices so you prove each signal catches its condition, and keep a healthy device as the negative control.
- **Widen in waves.** Fill `VPN Health – Windows – Production` gradually. Because the check is read-only there is no risk to the tunnels, but waves keep the telemetry and alert signal clean and interpretable.
- **Keep pilot and production distinguishable.** Use `VPNHEALTH – Tunnel Health – Detect – Pilot` and `… – Production` so you always know which assignment a result came from.

Telemetry & alerting

- **Verify the data path before trusting the dashboard.** Whether Proactive Remediations output is

natively queryable in Log Analytics — and via which table — is **verify-with-Microsoft**; confirm your health data actually lands in `LA – Endpoint – VPNHealth` and use that real table name in the KQL.

- **Alert on cohorts, not single devices.** One device dropping its tunnel for a minute is noise; a

cohort of devices on the same gateway failing to establish is a signal. Set `ALERT – VPN Tunnel Health – Degraded` thresholds so a cohort — or a cert-expiry window — triggers, not every transient.

- **Roll up by VPN type and gateway.** The `VPN Tunnel Health – Fleet Rollup` view by type and gateway

is what turns "some devices are unhealthy" into "gateway X for VPN type Y is the problem" — which points triage straight at the layer.

- **Watch the trend, not just the moment.** Export `VPNHEALTH-Report.ps1` regularly; a rising

Unhealthy/Degraded count for one VPN type or gateway is an early cohort signal.

Operate & govern

- **Name everything by intent and reuse verbatim.** Reuse `VPNHEALTH – Tunnel Health – Detect – Pilot`,

`PILOT – VPN Health – Devices`, `VPN Health – Windows – Production`, `LA – Endpoint – VPNHealth`, `ALERT – VPN Tunnel Health – Degraded`, and `VPN Tunnel Health – Fleet Rollup` across the console, scripts, testing, and troubleshooting so results are auditable.

- **Version the health-check script.** Keep `VPNHEALTH-Detect-TunnelHealth.ps1` under version control

with a changelog, especially the configuration block (probe targets, cert match, window), so you know exactly what "healthy" meant at any point.

- **Re-verify vendor and Microsoft specifics periodically.** VPN client detection surfaces, Proactive

Remediations behavior, Log Analytics data paths, and Azure Monitor wiring all drift — re-confirm the flagged items against current documentation on a schedule, not once.

- **Retire deliberately.** When a VPN technology is decommissioned, unassign and archive its monitor

rather than leaving a stale check reporting on tunnels that no longer exist.

8. Deliverables Summary — VPN Tunnel Health Monitoring & Alerting in Microsoft Intune (Scheduled Connectivity Health Checks, Telemetry & Alerts)

Everything included in this solution package, and what each piece is for.

Documentation (this guide)

- **Executive Summary** — the business case and outcome in one page.
- **Solution Overview** — the proactive endpoint-side monitoring model, the "define healthy" health

signals and thresholds, how it spans VPN types (Microsoft Tunnel, Always On device + user tunnel, third-party clients), the **explicit boundaries** (defers Microsoft Tunnel gateway-**server** health to the Tunnel Gateway product; defers VPN **deployment** to the deployment products; defers reactive break/fix to the Connectivity Toolkit; uses the Proactive Remediations detection model as its engine without reproducing it), benefits, and honest limitations. Emphasizes that the checks are **read-only**.

- **Requirements Guide** — every prerequisite (Proactive Remediations entitlement, an already-deployed

VPN profile, a Log Analytics workspace and Azure Monitor for telemetry/alerting, enrolled Windows devices, the Intune Management Extension, run-context, pilot and production groups, a Graph-capable admin workstation, and certificate visibility) with how to **verify** and how to **implement if missing**, plus the category-level Global Requirements. Licensing, data paths, alert wiring, and per-vendor specifics are flagged **verify-with-Microsoft**.

- **Implementation Guide** — exact, ordered, click-by-click steps across four phases: define the health

signals and author the read-only check, package and scope to a pilot, read results and wire telemetry (Log Analytics/KQL) and alerts (Azure Monitor), and validate then expand in waves. Every object is named and reused, and the fleet-rollup **KQL** is included.

- **Testing & Validation** — seven tests with explicit PASS/FAIL gates and an acceptance gate: the check

is proven **read-only**, the package reports **detection-only**, a **healthy device reports Healthy**, each **seeded break reports the correct red signal**, the **fleet rollup shows the cohort**, an **alert fires** on a seeded failure, and steady state shows no thrash.

- **Troubleshooting** — symptom → cause → fix for false red/false green, the script not running,

telemetry not arriving in Log Analytics, alerts not firing, per-VPN-type detection nuance, and a scripts section — plus the **Triage runbook** mapping each red signal to the responsible layer and the product that owns the fix.

- **Best Practices** — keep monitoring and fixing separate, define "healthy" well, deploy/scope,

telemetry & alerting, and operate/govern.

PowerShell scripts (device probe + Microsoft Graph) — `VPNHEALTH-Scripts.zip`

Each script has a header explaining **Purpose · How to run · Required scopes · Expected results**. Every device-side probe is **READ-ONLY** (never touches the tunnel); the single Graph state-changer deploys the **monitoring package only** and defaults to `-WhatIf`.

1. `VPNHEALTH-Detect-TunnelHealth.ps1` — the **read-only** device-side health check: verifies

adapter/profile present, tunnel connected, gateway reachable, key route + DNS through the tunnel, and certificate valid + expiry window; emits a structured verdict (Healthy/Degraded/Unhealthy) and the Proactive Remediations exit-code contract. VPN-type aware. **Makes no changes** — deploy it as the detection script with no remediation attached.

1. `VPNHEALTH-Discovery.ps1` — read-only (Graph): which VPN profiles are deployed, whether the

health-check package exists/is assigned, and a telemetry-path hint.

1. `VPNHEALTH-Validate.ps1` — PASS/FAIL that the package is deployed, assigned, running, reporting,

and **detection-only** (no remediation attached).

1. `VPNHEALTH-Report.ps1` — fleet health rollup export (per device: VPN type, verdict,

cert days-to-expiry) to CSV, from Graph or collected on-device logs; includes the equivalent Log Analytics **KQL** as a comment.

1. `VPNHEALTH-Deploy-HealthCheck.ps1` — the one state-changer (Graph): creates/assigns the

detection-only health-check package to the pilot group. **Defaults to** `-WhatIf` and says so. It deploys the **monitoring package only** — it never modifies a VPN.

Hero image

- `VPNHEALTH-hero.png` — the product image (1200×1200) used on the marketplace listing.

How it's delivered

A **Word** document and **PDF** (cover, table of contents, page numbers, and a scripts appendix), plus all five PowerShell scripts in a single **ZIP** — yours to keep and reuse across **unlimited devices**.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

VPNHEALTH-Deploy-HealthCheck.ps1

Purpose: The ONE state-changing script in this package. Via Microsoft Graph it creates the Proactive Remediations DETECTION-ONLY package 'VPNHEALTH - Tunnel Health - Detect - Pilot' from the local VPNHEALTH-Detect-TunnelHealth.ps1 and assigns it to the pilot group 'PILOT - VPN Health - Devices' on a daily schedule.

How to run: Install-Module Microsoft.Graph -Scope CurrentUser # once .\VPNHEALTH-Deploy-HealthCheck.ps1 # DRY-RUN (default) — shows the plan, no changes .\VPNHEALTH-Deploy-HealthCheck.ps1 -Execute # actually create + assign the monitor .\VPNHEALTH-Deploy-HealthCheck.ps1 -Execute -PilotGroupId <entra-group-guid>

VPNHEALTH-Detect-TunnelHealth.ps1

Purpose: A READ-ONLY device health check that evaluates a VPN tunnel against concrete health signals and returns a single verdict (Healthy / Degraded / Unhealthy) plus a per-signal breakdown. Signals: - VPN adapter/profile present - Tunnel established/connected - Gateway/endpoint reachable - Key route reachable THROUGH the tunnel - Internal DNS resolves THROUGH the tunnel - Auth certificate valid and not near expiry (default warning window: 30 days) It is VPN-TYPE AWARE (Microsoft Tunnel / Always On device tunnel / Always On user tunnel / third-party client such as Cisco Secure Client) and adapts what it probes per type.

How to run: This is UPLOADED to Intune as the DETECTION script of a Proactive Remediations package (Reports > Endpoint analytics > Remediations > Create — verify path with Microsoft docs). Leave the REMEDIATION script EMPTY — this is a detection-only monitor. Intune's management extension runs it on-device on the assignment's schedule. You can also run it locally on a pilot device to test the verdict. Run context: SYSTEM for Microsoft Tunnel / Always On DEVICE tunnel / most third-party checks; USER context for an Always On USER tunnel or a user-store certificate.

VPNHEALTH-Discovery.ps1

Purpose: Gives you a one-page picture, via Microsoft Graph, of what this monitoring practice has to work with and whether it is wired up: - Which VPN CONFIGURATION PROFILES are deployed in Intune (so you know what tunnels exist to monitor) and roughly what VPN technology each is. - Whether the health-check package 'VPNHEALTH - Tunnel Health - Detect - Pilot' (and the Production copy) EXISTS and is ASSIGNED. - Whether a Log Analytics / telemetry data path looks present (best-effort hint). Makes NO changes — read-only.

How to run: Install-Module Microsoft.Graph -Scope CurrentUser # once .\VPNHEALTH-Discovery.ps1 .\VPNHEALTH-Discovery.ps1 -NameLike 'VPNHEALTH'

VPNHEALTH-Report.ps1

Purpose: Exports a fleet health rollup to CSV so you have evidence of tunnel health over time. For each device reporting from the health-check package it captures the VPN type, the health verdict (Healthy / Degraded / Unhealthy), and the certificate days-to-expiry. Two data sources are supported: -Source Graph

(default): reads per-device run states + detection output from the Proactive Remediations package via Microsoft Graph (read-only), parsing the structured health line the detection script emits. -Source Local : parses on-device C:\ProgramData\VPNHealth\tunnel-health.log files you have collected (useful before the Log Analytics path is wired). Read-only. The equivalent Log Analytics KQL for the same rollup is included as a comment below.

How to run: Install-Module Microsoft.Graph -Scope CurrentUser # once .\VPNHEALTH-Report.ps1 .\VPNHEALTH-Report.ps1 -PackageName 'VPNHEALTH - Tunnel Health - Detect - Pilot' .\VPNHEALTH-Report.ps1 -CsvPath C:\Temp\vpnhealth.csv .\VPNHEALTH-Report.ps1 -Source Local -LocalLogFolder C:\Collected\VPNHealthLogs

VPNHEALTH-Validate.ps1

Purpose: Validates the acceptance gate for the health-check package (default 'VPNHEALTH - Tunnel Health - Detect - Pilot'): it EXISTS, is ASSIGNED (to at least one group, e.g. 'PILOT - VPN Health - Devices'), has run status REPORTING back from devices, and — as a best-effort hint — that a telemetry path is present. Produces a clear PASS/FAIL verdict. Read-only.

How to run: Install-Module Microsoft.Graph -Scope CurrentUser # once .\VPNHEALTH-Validate.ps1 .\VPNHEALTH-Validate.ps1 -PackageName 'VPNHEALTH - Tunnel Health - Detect - Pilot'